Technical Cheat Sheet — Tools, Commands & Syntax

Tools & libraries used

Tool/Library	Purpose
Python 3.10	Scripting language for ingestion
pandas	Reading/cleaning tabular data
pyarrow	Required for pandas to read `.parquet` files
snowflake-connector-python	Python ↔ Snowflake connection
python-dotenv	Loads credentials from `.env` safely
Snowflake	Cloud data warehouse
dbt-snowflake	dbt Core + Snowflake adapter, installed together
Looker Studio	BI/dashboard tool, free, connects to Snowflake
Git / GitHub	Version control, hosting the public repo

Terminal / bash commands

Navigation & files

```
pwd                          # show current directory
cd path/to/folder            # change directory
cd ..                        # go up one level
ls                           # list files
ls -a                        # list files including hidden (dotfiles)
ls -la                       # list with details (size, permissions, timestamp)
mkdir foldername             # create a folder
touch filename.ext           # create an empty file
open -e filename             # open a file in TextEdit
mv source destination        # move/rename a file (also used to overwrite)
```

Handling filenames with spaces/special characters

```
mv ~/Desktop/"File Name.png" newname.png      # quote the whole name
mv ~/Desktop/File\ Name.png newname.png       # escape spaces with backslash
mv ~/Desktop/File*10.17*.png newname.png      # wildcard to avoid typos entirely
```

Environment setup

```
pip install package_name                      # install a Python package
pip install package_name --break-system-packages   # needed on some
```

```
Mac setups
        python3 -m venv venv                              # create a virtual
environment
        source venv/bin/activate                           # activate it
(needed every new terminal session)
```

Git / GitHub commands

```
        git init                                  # initialize a repo in the
current folder
        git status                                 # see what's changed / staged
        git add .                                  # stage all changes
        git commit -m "message"                     # commit staged changes
        git push                                    # push to remote
        git push -u origin main                      # first push, sets upstream
tracking
        git branch -M main                            # rename current branch to
"main"
        git remote add origin <url>                    # connect local repo to a
GitHub repo
        git log --oneline                              # view commit history,
condensed
        git rm --cached path/to/file                    # untrack a file (keeps
it locally)
        git rm -r --cached foldername                     # untrack a whole
folder
        git config --global user.name "Name"               # set your git
identity
        git config --global user.email "you@email.com" # set your git email
```

.gitignore basics

```
.env                    # ignore this exact file
data/*.parquet           # ignore all .parquet files in data/
logs/                     # ignore an entire folder
```

GitHub authentication note

GitHub no longer accepts your regular account password for `git push`. When prompted for a password, you need a **Personal Access Token** instead: 1. github.com/settings/tokens → Generate new token (classic) 2. Check the `repo` scope 3. Copy the token, paste it as the “password” when git prompts you

---

Python — ingestion pattern (the shape of `ingest.py`)

```
        import pandas as pd
        import snowflake.connector
        from dotenv import load_dotenv
        import os

        load_dotenv()                                      # loads .env into
environment variables

        df = pd.read_parquet("data/file.parquet")         # read raw data
        df.columns = [c.lower().strip() for c in df.columns]   # clean
```

```
column names
        df = df.dropna(subset=["col1", "col2"])             # drop rows
missing key fields
        df = df[df["some_col"] > 0]                          # filter bad
values
        df["timestamp_col"] = df["timestamp_col"].astype(str)  # avoid
Snowflake timestamp binding errors

        conn = snowflake.connector.connect(
            account=os.getenv("SNOWFLAKE_ACCOUNT"),
            user=os.getenv("SNOWFLAKE_USER"),
            password=os.getenv("SNOWFLAKE_PASSWORD"),
            warehouse=os.getenv("SNOWFLAKE_WAREHOUSE"),
            database=os.getenv("SNOWFLAKE_DATABASE"),
            schema=os.getenv("SNOWFLAKE_SCHEMA"),
        )
        cursor = conn.cursor()

        # batch insert (10,000 rows at a time)
        batch_size = 10000
        for i in range(0, len(df), batch_size):
            batch = df.iloc[i:i+batch_size]
            rows = [tuple(r) for r in batch.itertuples(index=False,
name=None)]
            cursor.executemany("INSERT INTO table (col1, col2) VALUES (%s,
%s)", rows)

        cursor.close()
        conn.close()
```

Handling missing values before insert (avoids `invalid identifier 'NAN'` errors):

```
        df = df.where(pd.notnull(df), None)   # converts NaN to Python None
/ SQL NULL
```

Snowflake SQL syntax

Setting up structure

```
        CREATE DATABASE IF NOT EXISTS MY_DB;
        CREATE SCHEMA IF NOT EXISTS MY_DB.RAW;
        CREATE OR REPLACE TABLE MY_DB.RAW.MY_TABLE (
            col1 VARCHAR,
            col2 TIMESTAMP,
            col3 FLOAT
        );
```

Common queries used

```
        SELECT COUNT(*) FROM table_name;                  -- row count
        SELECT * FROM table_name LIMIT 10;                  -- peek at data
        SELECT MIN(date_col), MAX(date_col) FROM table;       -- check date
range (found bad 2008 date this way)
        TRUNCATE TABLE table_name;                            -- clear all
rows, keep structure
```

Useful Snowflake date/time functions

```
        DAYNAME(timestamp_col)          -- returns 'Mon', 'Tue', etc.
```

```
        HOUR(timestamp_col)              -- returns 0-23
        DAY(date_col)                     -- returns day-of-month as integer
        CAST(col AS DATE)                  -- convert timestamp to date
        CAST(col AS TIMESTAMP)              -- convert to timestamp
        DATEDIFF('minute', col1, col2)       -- difference between two
timestamps
        CURRENT_TIMESTAMP()                   -- right now
```

---

dbt commands

```
        dbt init                          # scaffold a new dbt project,
connect to warehouse
        dbt debug                          # test the connection defined in
profiles.yml
        dbt run                             # build all models
        dbt run --select model_name           # build just one model
        dbt test                              # run all tests
        dbt test --select model_name            # run tests for one model
only
```

profiles.yml structure (lives in ~/.dbt/profiles.yml, not in the project repo)

```yaml
        project_name:
          target: dev
          outputs:
            dev:
              type: snowflake
              account: your_account_identifier
              user: your_username
              password: your_password
              role: ACCOUNTADMIN
              database: YOUR_DB
              warehouse: COMPUTE_WH
              schema: STAGING
              threads: 4
```

Basic model file structure (a .sql file in models/)

```
        with source as (
            select * from {{ ref('other_model') }}      -- reference another
dbt model
            -- or: select * from RAW_DB.RAW.RAW_TABLE   -- reference a raw
table directly
        ),

        cleaned as (
            select
                col1,
                cast(col2 as date) as col2,
                count(*) as row_count
            from source
            group by col1, col2
        )

        select * from cleaned
```

schema.yml — defining tests

```yaml
version: 2

models:
  - name: model_name
    columns:
      - name: column_name
        tests:
          - not_null
          - unique
      - name: another_column
        tests:
          - accepted_values:
              arguments:
                values: ['1', '2', '3']
      - name: foreign_key_column
        tests:
          - relationships:
              to: ref('other_model')
              field: id_column_in_other_model
```

Custom singular test (a standalone .sql file in tests/)

A singular test is just a query — if it returns any rows, the test fails.

```sql
-- tests/my_custom_test.sql
select *
from {{ ref('model_name') }}
where some_column < 0
   or (another_column > 500 and yet_another < 5)
```

dbt_project.yml — model config (avoiding deprecation warnings)

```yaml
models:
  your_project_name:
    staging:
      +materialized: view
    marts:
      +materialized: view
```

Looker Studio setup notes

Connecting to Snowflake: - Add data → Snowflake connector → enter account/user/password/warehouse/role/database/schema - Use a custom SQL query per chart's data source: `select * from DB.SCHEMA.TABLE_NAME`

Displaying one field but sorting by another (e.g. day names in calendar order): 1. Set the chart's Row/Column **dimension** to the readable field (e.g. `DAY_OF_WEEK`) 2. In the chart's **Sorting** section, set the sort field to a separate numeric helper column (e.g. `DAY_ORDER`), ascending 3. This displays "Mon, Tue, Wed…" while actually sorting by 1, 2, 3…

Getting a shareable link: Share (top right) → Get link → change to "Anyone with the link can view" → copy link

---

Quick reference: the full pipeline order of operations

1. `python3 ingest.py` — loads raw taxi trips into Snowflake `RAW` schema
2. `python3 load_zones.py` — loads the zone lookup table into Snowflake `RAW` schema
3. `cd taxi_project && dbt run` — builds all staging and marts models in dependency order
4. `dbt test` — validates data quality across all models
5. Looker Studio pulls from the `STAGING` schema mart tables to power the dashboards
6. `git add . && git commit -m "..." && git push` — saves and publishes changes